Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

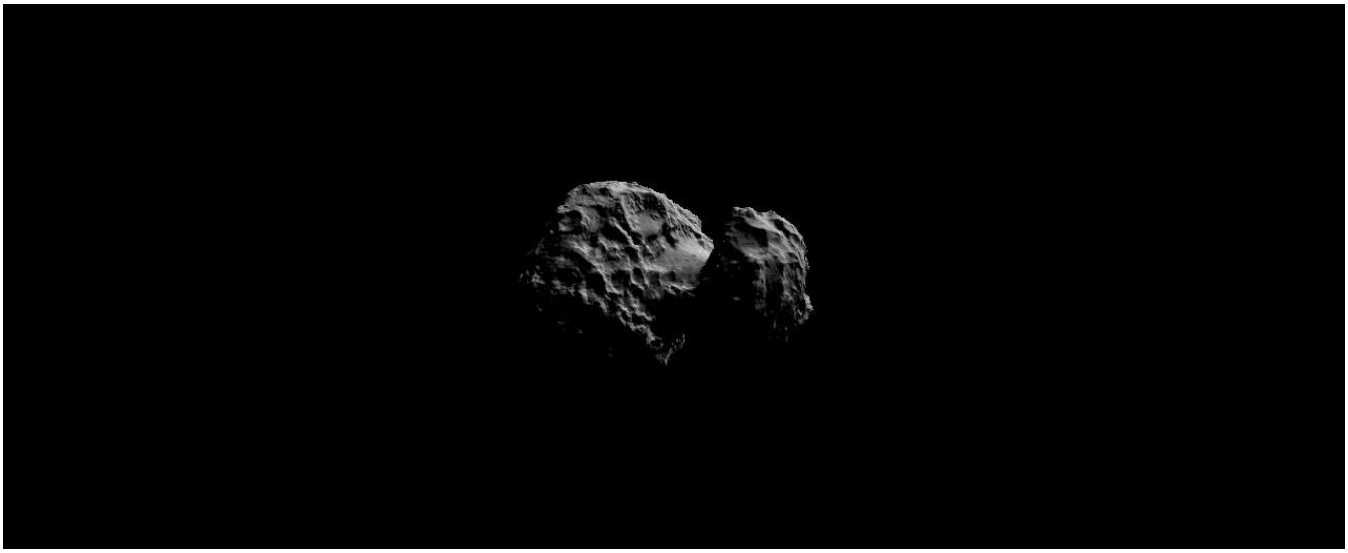
Jun 1, 2020 · 10 min read · ✦ · Listen

Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — A comet in 3 D

Part 12 of the tutorial series will start with a scientific view on a particular comet: 67P/Churyumov–Gerasimenko



A rendered 3 dimensional model of the comet 67P/Churyumov–Gerasimenko. How can one understand, work and visualise such nice shape models? Well, that's the goal of our lesson today. Credit: T. Albin

Preface

This is the 12th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

6 | 1

(A short & personal) Introduction

It's the 12th November 2014. The last couple of days were overwhelming. So overwhelming that I did not have much sleep … now … I am waiting, sitting in front of screens with several other scientists to wait and witness a scientific and engineering milestone in human history: The landing of a spacecraft on the comet 67P/Churyumov–Gerasimenko. The lander's name: Philae; an essential part of **ESA's** mission **Rosetta/Philae**.

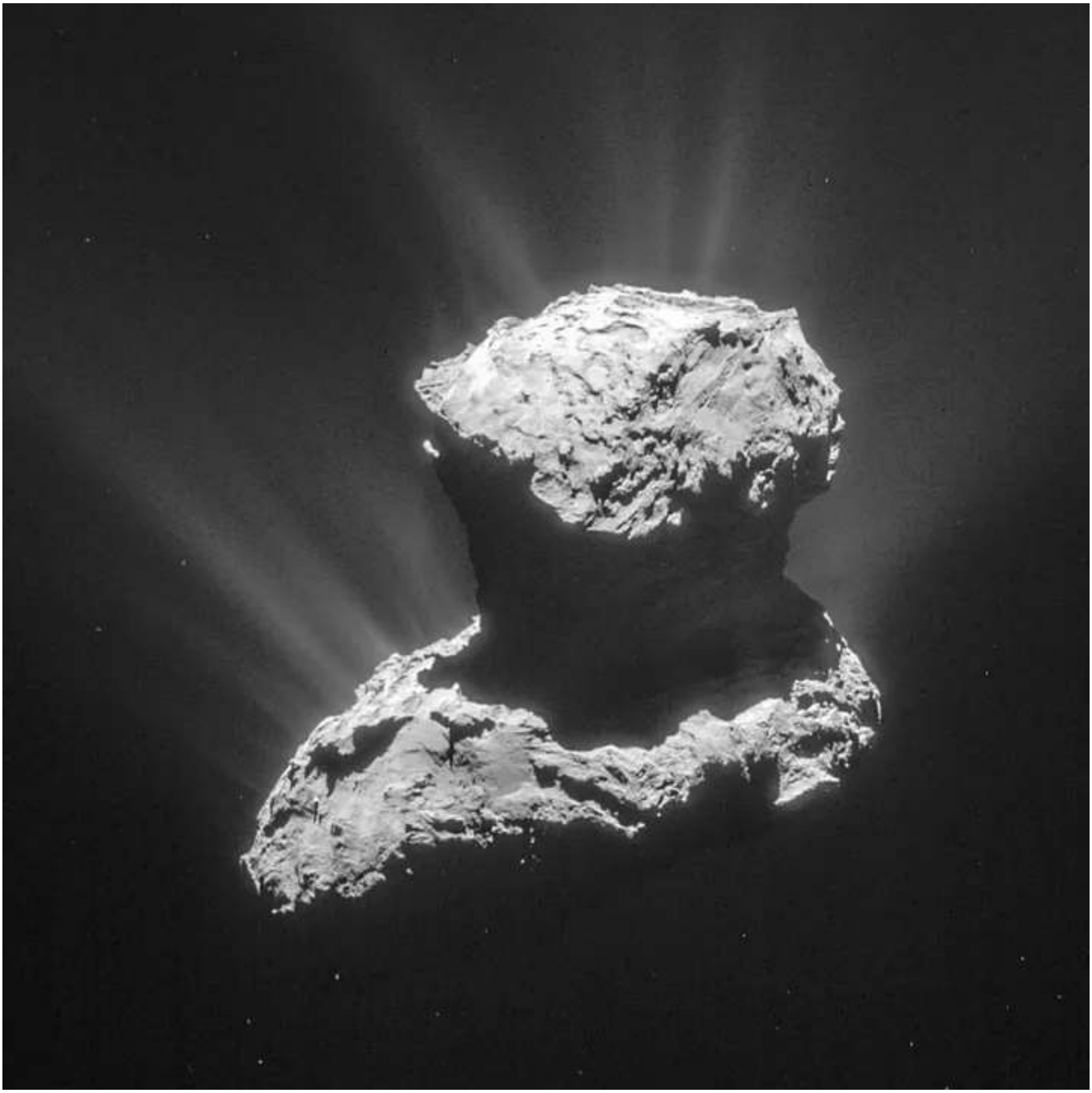
Trajectory of the Rosetta spacecraft through interplanetary space. This animation uses the Rosetta spk SPICE kernels to compute the path of the spacecraft and celestial objects. Today, we will have a first look at the library that was used to create this animation. Credit: T. Albin

Spacecraft missions are indeed generation projects. I have seen documents of the mission's ideas and objectives that were dated back when I was visiting elementary school. During secondary education the mission launched (2004), during by Bachelor's studies the mission was still flying in interplanetary space (2009–2012). Then, during my Master's in 2014, I was sitting now with scientists in the lander control centre in Cologne (Germany), working in the Dust Impact Monitor (DIM) team, as a part of the Surface Electric Sounding and Acoustic Monitoring Experiment (SESAME) collaboration [1, 2, 3]. DIM was a rubric's cube sized and shaped instrument on the lander. It's objective was to measure impacts of millimetre sized dust particles during the landing (descent) phase and on the comet's surface itself. The measurement principle was simple: DIM analysed and

derived the particles' properties from impacts on piezo-electric elements … these are the same electronic components that convert swinging guitar strings to electric signals … making it an *electric guitar*.

The landing of Philae and months of close observations with the Rosetta orbiter revealed a *ancient* cosmic world formed at the very beginning of our Solar System: consisting of water ice, minerals and an abundance of organic compounds. Rosetta/Philae, and my personal story that is linked to it would probably be a nice addition for another less data scientific Medium article. For now, what matters is that the mission gathered a huge amount of data that is analysed or still waiting to be analysed and also published. All data are publicly available, either on NASA's Planetary Data System's site (PDS) and / or on ESA's Planetary Science Archive (PSA).

In the last sessions, we learned a lot about comets, their origin, their relation to Jupiter and observational biases. Today, we will have a look at the shape of this exemplary comet core.

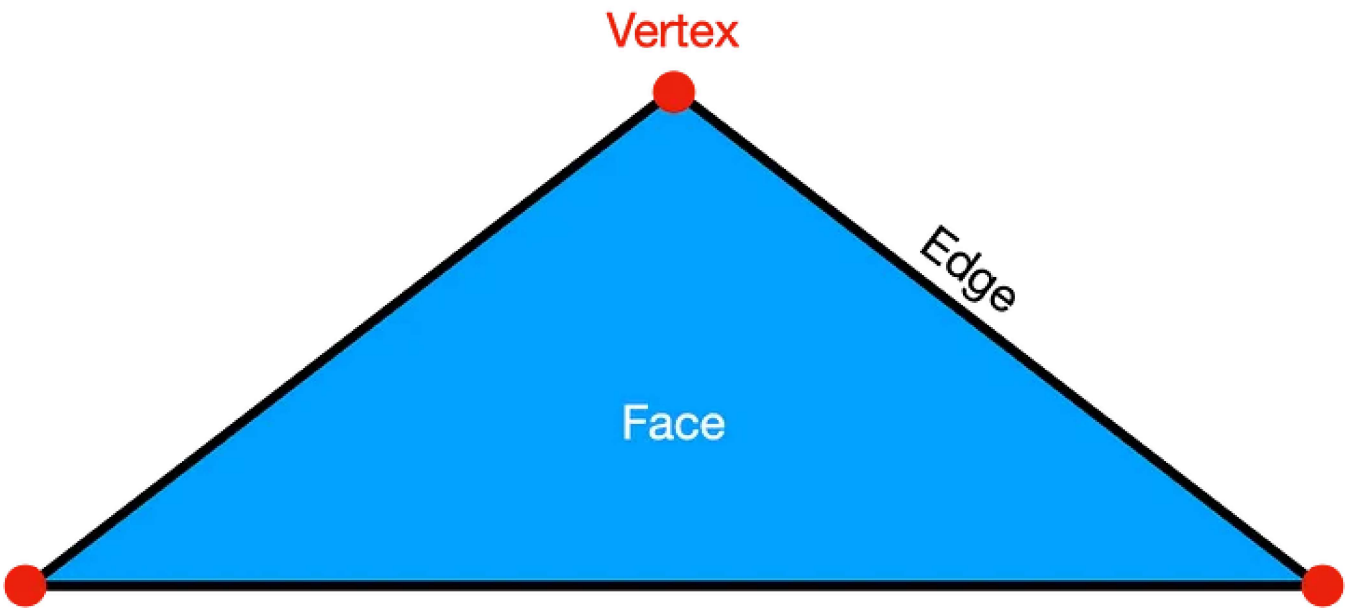


Did comets bring essential life “building blocks” to Earth? Maybe. 67P/Churyumov–Gerasimenko (image) does contain crucial organic compounds. The white “stripes” that leave the comet are dust and gas streams. Credit: ESA/Rosetta/NavCam; License: CC BY-SA IGO 3.0

Shape Models

Preusker et al. (2015) [1] used the image data from a camera system on board the orbiter Rosetta called *Optical, Spectroscopic and Infrared Remote Imaging System* (OSIRIS) to define a coordinate system for this weirdly formed comet as well as to derive a so called shape model. A shape model contains 3 dimensional information of an object that allows one to render, visualise and work with 3 D objects. Shape models contain so called *vertices* and additionally *edge* or *face* information:

- **Vertex**: A vertex is simply a positional vector that contains the X, Y and Z coordinates of a point in space
- **Edge**: Edges link vertices together. They contain two index information of the corresponding vertices that are connected
- **Face**: Edges enclose areas that are called faces. Instead of defining edges one can define a list of vertex indices that create a face (e.g., a triangle, rectangle, etc.)



Convention of a shape model's triangle (example). Credit: T. Albin

Shape models of the comet 67P have also been derived by the so-called *Navigation Cameras* (NAVCAM) that were used to determine the spacecraft's orientation in space. NAVCAM was intentionally an engineering device that allowed one however to derive scientific insights.

NASA's mighty SPICE toolkit that we used in the previous tutorial session has some great functionalities to work with shape models. These models are abbreviated as so-called *dsk*s (digital shape kernels). We will get back to the *dsk* functions in the future.

A 3 dimensional view on 67P

Today, we will need to install a new package for 3 D rendering: *visvis*. *visvis* works with miscellaneous graphical backends like: FLTK, GTK, Wx and also PyQT5. For this tutorial we will be working with QT and install it via:

```
pip install PyQt5
```

Now we can install *visvis* via:

```
pip install visvis
```

Please note a few things: The installation and testing for this session has been done on Mac OS Catalina. If you face any issues with Linux or Windows, do not hesitate to ask here. We will try to figure it out.

The creator of *visvis* provides a nice wiki page if you are interested in more examples. The creator also states that there are similar libraries out there like *vispy*.

First, let's import the already known libraries (also *imageio* and *tqdm* as used in a supplementary article). We will also need our custom function *func* (line 10) to download the shape model. Due to its size of around 85 MB the shape model cannot be stored on the tutorial corresponding GitHub repository.

An extra folder (line 2) shall be created for the shape model. The download link of the model is listed in line 3 and 4. You can see that the link contains the *dsk* SPICE notation. The file name of the shape model appears somehow cryptic so let's take a look at the folder's corresponding readme file. It states:

```
ROSETTA Spacecraft DSK Naming Convention for the science phase:

                PPP_BB_RRRR_IMETPRO_N_VV.BDS
```

With the abbreviations:

```
token          description
------------   ---------------------------------------------
PPP            project prefix: ROS

BB             body: CG for 67P/C-G, LU for Lutetia, or ST for
               Steins

RRRR           number of plates: Knnn (for nnn thousand plates) or
               Mnnn (for mmm million plates)

IMETPRO        camera-method-producer combined into a single
               7-character token:

               I   -  camera:

                 O for OSIRIS
                 N for NAVCAM
                 M for multiple

               MET -  method:

                 SPC for StereoPhotoClinometry
                 MSD for Multi-Resolution StereoPhotoClinometry
                 SPG for StereoPhotoGranulometry

               PRO -  producer:

                 ESA for European Space Agency/ESOC
                 LAM for Laboratoire d'Astrophysique de Marseille
                 LPS for Laboratoire d'Astrophysique de Marseille
                         + Planetary Science Institute
                 DLR for Deutsches Zentrum fur Luft- und Raumfahrt
                         e.V.
```

```
N               surface naming: U for unnamed (surface ID = body ID,
                for use with Toolkits N0065 or earlier) or N for
                named (surface IDs are set to distinct numbers that
                are mapped to surface names using keywords provided
                in the project FK v25 or later)

VV              version: V1, V2, V3 ....

BDS             extension: .BDS
```

In our case, we have a shape model of the comet 67P/Churyumov–Gerasimenko (*CG*) from the Rosetta mission (*ROS*). There are at least 1 million plates (faces) in the data set (*M0001*), derived from the OSIRIS camera (*O*) with a so-called Stereo Photo Clinometry (*SPC*) method by the Laboratoire d'Astrophysique de Marseille + Planetary Science Institute (*LPS*): *OSPCLPS*. The object is named (*N*) and it is currently version 1 (*V1*). The data are stored in a so called *OBJ* file.

Part 2/10

After the download, we load the shape model with the *pandas* *read_csv* function. The file has 4 columns. The first one indicates the type with a *v* for vertex or *f* for face. The last three columns are the corresponding coordinates (vertex) or vertex indices (face). According to the printed results (see below) we have almost 600,000 vertices and over one million faces, as the file name already indicated.

Part 3/10

```
Some statistics and parameters of the shape model
Rows and columns of the data set: (1791747, 4)
```

```
Number of vertices: 597251
Number of faces: 1194496
```

Now, we can take a look at the vertex (line 2 and 3) and face (line 4 and 5) data, by printing the first 5 rows with the *pandas* *head* function:

Part 4/10

We can see the coordinates for the vertices and the indices for the vertices (faces). The coordinates of the vertices are given in km and represent the actual physical 3 dimensional size of the comet's core!

```
Vertices (Sample)
  TYPE        X1        X2        X3
0    v  0.570832 -1.000444  0.532569
1    v  0.564360 -1.000224  0.525360
2    v  0.557853 -0.997863  0.520762
3    v  0.553592 -0.998414  0.512192
4    v  0.550212 -0.992514  0.507304
Faces (Sample)
       TYPE      X1      X2      X3
597251    f   474.0   522.0   305.0
597252    f   474.0   719.0   522.0
597253    f  1961.0  2160.0  1651.0
597254    f  1961.0  2159.0  2160.0
597255    f  5513.0  5668.0  5285.0
```

We can easily assign a Python list with the vertices (line 2 and 3), by converting the vertices sub-set to a list with *tolist*. The faces however (line 4), need some more data parsing. We will see it in the next code snippet.

Part 5/10

Let's print the minimum (line 2) and maximum (line 3) vertex index in the faces sub-set …

Part 6/10

… We see, that the minimum index is 1; further, the index appears to be a float. For our Python routine, the indices start at 0. We need to subtract the indices by 1:

```
Minimum vertex index in faces: 1.0
Maximum vertex index in faces: 597251.0
```

The subtraction is simply done in line 2. Afterwards, we convert the floats to integers and convert the faces sub-set to a Python list.

Part 7/10

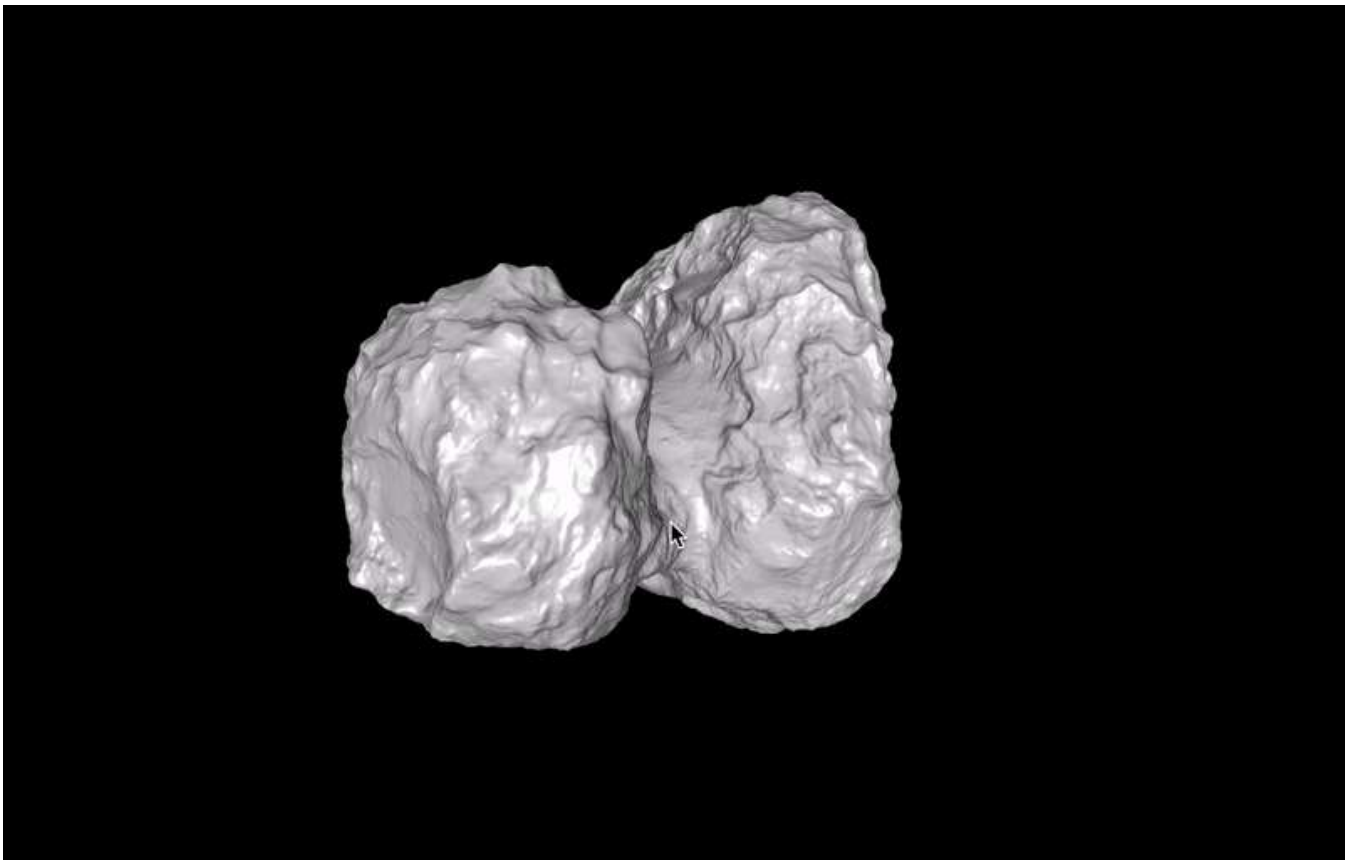
We are now ready to go! We should be able to adjust the resolution of the window, to change the settings according to the computer's performance. For this purpose, the creator of *visvis* provides a nice example for QT4. We need to change some imports for QT5:

Part 8/10

And we are ready to explore our comet! First, we create the *visvis* application (line 2 and 3). We create the main window that was defined before in line 6 and set a proper resolution in line 7. Now, with the function *mesh* the shape model is read. The vertices and faces are committed as well as a parameter that tells the function how many vertices define a face (here: 3). Line 14 gets the axes objects (similar as in *matplotlib*) that allows us to set a background colour (line 17) and to adjust some parameters (line 20 and 21). Now, we set a camera setting (*3d*) in line 26. Note: if you want to "fly" along the comet, like in a game (with w-a-s-d (translation) and i-j-k-l (tilt)), replace this parameter with "fly". The field-of-view is defined in 29 as well as the initial camera's azimuth and elevations angle (line 32 and 33). And finally, we can run it (line 36).

Part 9/10

Below, you can see a short movie. Feel free to run the application locally on your computer, adjust the settings and explore this cosmic world. 67P is weirdly shaped. It consists much likely of a binary system that merged at some point in the past. The "rubber duck" consists of two bodies with crater and basins, while the "neck" in between is much smoother. The object (486958) Arrokoth has a similar shape that was being visited by NASA's New Horizons mission.

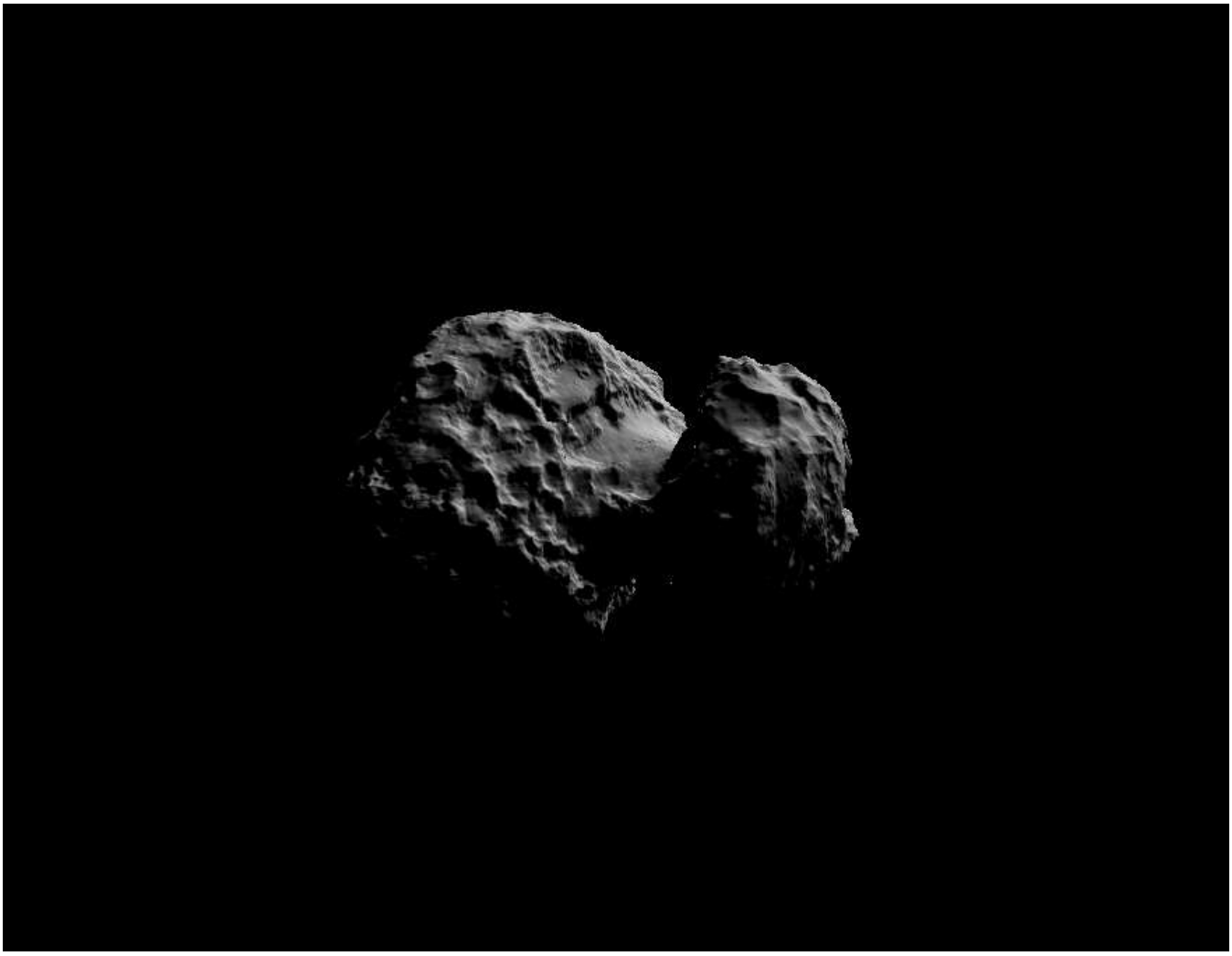


"Playing around" and exploring the comet 67P. Left-Click: Rotate the comet. Scroll wheel: Zoom in and out. Shift+Left-Click: Change the camera's pointing spot. Double Left-Click: Default camera reset. Credit: T. Albin

Finally, let's finish our exploration with the very first animation at the beginning of the tutorial. For this purpose, we set a smaller resolution, since the resulting GIF should not exceed 25 MB (Medium limit; line 9). We change the material properties of the comet in line 14 and 15; the resulting object is less shiny and appears more realistic. Further, per default, our camera view has an "attached spotlight" that illuminates the object. We turn this light source off (line 36 and 37) and add a fixed light source in line 38. Now, in line 44 to 57 we change the camera's azimuth angle dynamically in a for-loop (line 47) and re-draw the figure in each step (line 50 and 51). The resulting image is extracted in line 54 and line 57 converts the image data to

an 8 bit matrix and appends the result to a list that was defined in line 41. Finally, in line 60, we create the animation as a GIF.

Part 10/10



We finish today's session with the final shape model visualisation from the beginning. This visualisation results from code snippet 10/10. Credit: T. Albin

Conclusion

Enjoy the 3 D power of Python and *visvis*! Scientists have worked hundreds of hours to derive this scientifically highly accurate shape model after waiting for years to reach the comet! It is now up to you to explore this world and compare the model with actual observations stored in the ESA image database. We will work with this model and derive some more parameters like the entire surface of the comet and we will also work with the coordinate system of 67P. *What do we know about 67P?*

Thomas

Appendix

By the way. The methods that I am showing have actually been used for scientific insights and publications! Take a look at [2]: Link. In figure 3, you can see the recent

model back then. With the Philae corresponding spk kernel you could try to reproduce this figure (without the lander)!

References

[1] Flandes, Alberto; Krüger, Harald; Loose, Alexander; Albin, Thomas; Arnold, Walter. *Dust Impact Monitor (DIM) onboard Rosetta/Philae: Tests with ice particles as comet analog materials*. Planetary and Space Science, Volume 99, p. 128–135. September 2014. DOI: 10.1016/j.pss.2014.05.014

[2] Krüger, Harald; Seidensticker, Klaus J.; Fischer, Hans-Herbert; Albin, Thomas; Apathy, Istvan; Arnold, Walter; Flandes, Alberto; Hirn, Attila; Kobayashi, Masanori; Loose, Alexander; Péter, Attila; Podolak, Morris. *Dust Impact Monitor (SESAME-DIM) measurements at comet 67P/Churyumov-Gerasimenko*. Astronomy & Astrophysics, Volume 583, id.A15, 13 pp. November 2015. DOI: 10.1051/0004–6361/201526400. arXiv: arXiv:1510.0156

[3] Hirn, Attila; Albin, Thomas; Apáthy, István; Della Corte, Vincenzo; Fischer, Hans-Herbert; Flandes, Alberto; Loose, Alexander; Péter, Attila; Seidensticker, Klaus J.; Krüger, Harald. *Dust Impact Monitor (SESAME-DIM) on board Rosetta/Philae: Millimetric particle flux at comet 67P/Churyumov-Gerasimenko*. Astronomy & Astrophysics, Volume 591, id.A93, 7 pp. June 2016. DOI: 10.1051/0004–6361/201628370. arXiv: arXiv:1605.06291

[4] Preusker, F.; Scholten, F.; Matz, K. -D.; Roatsch, T.; Willner, K.; Hviid, S. F.; Knollenberg, J.; Jorda, L.; Gutiérrez, P. J.; Kührt, E.; Mottola, S.; A'Hearn, M. F.; Thomas, N.; Sierks, H.; Barbieri, C.; Lamy, P.; Rodrigo, R.; Koschny, D.; Rickman, H.; Keller, H. U. Agarwal, J.; Barucci, M. A.; Bertaux, J. -L.; Bertini, I.; Cremonese, G.; Da Deppo, V.; Davidsson, B.; Debei, S.; De Cecco, M.; Fornasier, S.; Fulle, M.; Groussin, O.; Güttler, C.; Ip, W. -H.; Kramm, J. R.; Küppers, M.; Lara, L. M.; Lazzarin, M.; Lopez Moreno, J. J.; Marzari, F.; Michalik, H.; Naletto, G.; Oklay, N.; Tubiana, C.; Vincent, J. -B. *Shape model, reference system definition, and cartographic mapping standards for comet 67P/Churyumov-Gerasimenko — Stereo-photogrammetric analysis of Rosetta/OSIRIS image data*. Astronomy & Astrophysics, Volume 583, id.A33, 19 pp. November 2015. DOI: 10.1051/0004–6361/201526349

Python Science Space Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play